

SOLIDA

Registre des formules de calcul

Dernière mise à jour : 7 août 2026

Objet de ce document

Ce document répertorie **toutes** les formules de calcul utilisées par SOLIDA pour produire un score, une décision, un plafond de crédit ou une échéance – sans exception. Pour chacune, il précise : l'écran où le résultat apparaît, chaque variable et chaque constante en jeu avec sa valeur actuelle, la formule telle qu'elle est implémentée aujourd'hui, pourquoi elle a été choisie ainsi (y compris quand la réponse honnête est « aucune source, choix arbitraire »), et ce que le modèle de scoring réel changera – ou ne changera pas – une fois entraîné.

C'est la référence unique pour comprendre ou modifier un calcul sans relire le code source. Le code lui-même ne porte que des renvois courts vers ce document : le détail et la justification vivent ici, pas dans des commentaires longs qui alourdissent la lecture du code.

Principe transversal, valable pour toutes les formules ci-dessous sauf mention contraire : le modèle de scoring réel, une fois entraîné, améliore uniquement la probabilité de défaut p qu'il produit. Il ne calibre jamais tout seul les paramètres de politique de risque (marges, seuils, coefficients de modulation) qui transforment ensuite cette probabilité en score, en décision ou en plafond – ce sont des arbitrages de la coopérative, qui se calibrent par rétro-test sur des cohortes réelles de remboursement, pas par l'entraînement du modèle. Chaque section précise l'exception quand elle existe.

Origine des valeurs actuelles. Une partie des constantes ci-dessous (grille de décision, crédit progressif, conditions de réexamen) proviennent d'un prototype de calcul, `simulateur/decision.py`, écrit pour démontrer le mécanisme avant que le vrai backend n'existe, puis retiré du dépôt une fois son rôle de démonstration rempli (trace complète dans `03-MODELE/09-lecons-prototype-simulateur.md`). Ce prototype n'a jamais prétendu être calibré sur des données réelles : c'est un point de départ illustratif, pas une vérité mesurée. Chaque section indique explicitement quand une valeur a cette origine, et quand elle n'a **aucune** source documentée du tout.

Table des matières

1	Transformation probabilité → score (échelle PDO)	3
2	Invariant de cohérence de la décomposition	5
3	Décomposition en points (« Facteurs déterminants »)	6
4	Contributions factices du modèle de substitution	7
5	Grille de décision (tranche)	9
6	Plafond de crédit progressif	11

7 Palier accessible au prochain cycle	13
8 Conditions de réexamen	14
9 Cascade : choix du mode de calcul (socle vs enrichi)	15
10 Tendance de l'épargne	16
11 Échéance mensuelle et taux d'endettement	17
12 Politique de mot de passe	18

1 Transformation probabilité $\rightarrow$ score (échelle PDO)

Écran(s) concerné(s) : Résultat de scoring (jauge et nombre de points), fiche de justification.

Implémentation : `backend/solida/domain/rules/scorecard.py::calculer_score`

Variables

Symbole	Signification
p	Probabilité de défaut produite par le modèle de scoring

Constantes

Constante	Valeur	Pourquoi cette valeur
PDO (points pour doubler les cotes)	20	Reprend l'exemple canonique de la littérature scorecard (popularisé par les manuels de <i>credit scoring</i> et repris comme valeur par défaut dans la plupart des outils de scorecard), pas une calibration propre à SOLIDA.
$score_ref$	600	Idem : point d'ancrage canonique de la même littérature.
$odds_ref$	50	Idem : rapport de cotes canonique associé à $score_ref = 600$ dans le même exemple de référence.
$score_min$	300	Convention du score FICO américain (300–850), reprise pour sa familiarité, sans lien avec la distribution réelle du futur modèle.
$score_max$	850	Idem.

Formule

$$Facteur = \frac{PDO}{\ln 2} \quad (1)$$

$$Décalage = score_ref - Facteur \times \ln(odds_ref) \quad (2)$$

$$score = \text{clip} \left(Décalage + Facteur \times \ln \left(\frac{1-p}{p} \right), score_min, score_max \right) \quad (3)$$

Convention bancaire : score élevé = risque faible, d'où le rapport $(1-p)/p$ et non son inverse.

Pourquoi cette formule

C'est la transformation PDO standard de la scorecard bancaire (logistique $\rightarrow$ score linéaire), qui garantit qu'une variation constante du score correspond toujours au même facteur multiplicatif sur le rapport de cotes, quel que soit le niveau de risque de départ. C'est ce qui rend le score lisible et comparable d'un dossier à l'autre.

Statut et rôle du modèle réel

Exception au principe transversal : contrairement à la grille et au crédit progressif, PDO , $score_ref$ et $odds_ref$ doivent être recalibrés dès que le modèle réel existe – l'échelle n'a

de sens que rapportée à la distribution réelle de p que ce modèle produit. Un score mal calibré resterait affiché de façon trompeuse même si la décision sous-jacente (section 5) reste correcte, puisqu'elle raisonne directement sur p , jamais sur le score affiché. *score_min* et *score_max* n'ont pas besoin de changer : ce sont de simples bornes d'affichage.

2 Invariant de cohérence de la décomposition

Écran(s) concerné(s) : Aucun – vérification interne avant l’enregistrement de toute décision.

Implémentation : `backend/solida/domain/rules/scorecard.py::verifier_invariant_decomposition`

Constante

Constante	Valeur	Pourquoi cette valeur
TOLERANCE_ INVARIANT	1,0 point	Marge technique pour l’arrondi flottant accumulé en sommant plusieurs contributions, pas un seuil métier : assez large pour ne jamais déclencher sur un arrondi légitime, assez serrée pour rejeter toute vraie incohérence de calcul.

Formule

$$\left| points_de_base + \sum_j points_j - score \right| < TOLERANCE_INVARIANT$$

Si l’inégalité est violée, le scoring est **rejeté** (exception `InvariantScoreViole`) plutôt que rendu de façon incohérente.

Pourquoi ce mécanisme

Une fiche de justification dont les points ne somment pas exactement au score détruirait la crédibilité du produit devant un sociétaire ou un auditeur. Mieux vaut refuser un scoring que l’afficher faux.

Statut et rôle du modèle réel

Aucun changement attendu : cette vérification reste valable quelle que soit la source des contributions, apprises ou factices.

3 Décomposition en points (« Facteurs déterminants »)

Écran(s) concerné(s) : Résultat de scoring (graphique « Facteurs déterminants »), fiche de justification.

Implémentation : `backend/solida/domain/rules/scorecard.py::decomposer_en_points`

Variables

Symbole	Signification
β_0	Constante (log-cotes de base, hors effet des variables)
f_j	Contribution en log-cotes de la variable j
<i>Facteur, Décalage</i>	Identiques à la section 1

Formule

$$points_de_base = Décalage + Facteur \times \beta_0 \quad (4)$$

$$points_j = Facteur \times f_j \quad (5)$$

Pourquoi cette formule

La transformation probabilité $\rightarrow$ score étant affine, chaque terme du log-cotes se met à l'échelle indépendamment par le même facteur : la décomposition en points hérite donc exactement des mêmes propriétés que le score global, ce qui rend l'invariant de la section 2 possible.

Statut et rôle du modèle réel

Le modèle réel doit fournir des f_j qui soient de véritables contributions en log-cotes (native-ment additives pour un modèle de type EBM/GAM, ou approximées par des valeurs de Shapley pour un autre type de modèle). Voir section 4 pour l'état actuel de β_0 et des f_j , aujourd'hui entièrement fictifs.

4 Contributions factices du modèle de substitution

Écran(s) concerné(s) : Résultat de scoring (graphique « Facteurs déterminants »), fiche de justification.

Implémentation : `backend/solida/adapters/ml/modele_constant.py::ModeleConstant.contributions` ; `backend/solida/application/use_cases/scorer_demande.py` (calcul de `beta_0`)

Contexte

`ModeleConstant` (le modèle de substitution utilisé tant qu’aucun modèle réel n’est entraîné) renvoie une probabilité **fixe** $p = 0,09$ – le taux de créances en souffrance cible du générateur de données, pas une valeur inventée. Comme p ne varie jamais avec le dossier, β_0 est recalculé à chaque scoring pour absorber exactement l’écart :

$$\beta_0 = \ln\left(\frac{1-p}{p}\right) - \sum_j f_j$$

Conséquence mathématique importante : **le score et la décision ne dépendent jamais** de ce que `contributions()` renvoie, quelle que soit sa valeur. C’est ce qui rend sûr d’y mettre une décomposition non apprise sans risquer de fausser une décision réelle.

Constantes – coefficients choisis à la main, pas appris

Variable	Coefficient	Point de référence
Régularité des dépôts (12 mois)	0,12	6 mois
Taux d’endettement	−1,4	0,35
Nombre d’incidents antérieurs	−0,5	0
Ancienneté sociétaire (mois)	0,01	36 mois
Épargne rapportée au revenu	0,6	0,3
Épargne rapportée au montant demandé	0,5	0,5

Pourquoi ces valeurs précises

Aucune source, aucune calibration. Seul le *signe* de chaque coefficient reprend l’intuition métier attendue (même intuition que `simulateur/config/config.yaml`, jamais ses valeurs). Les points de référence sont choisis pour être des valeurs médianes plausibles d’un dossier moyen, à l’œil, sans mesure. C’est une démonstration délibérément assumée comme non apprise, pas une estimation.

Formule (par variable)

$$f_j = \text{coefficient}_j \times (\text{valeur}_j - \text{référence}_j)$$

Pourquoi cette fonction existe

Sans elle, l’écran « Facteurs déterminants » afficherait une liste vide (« Aucune contribution significative identifiée »), ce qui est correct mais peu utile pour juger l’interface avant qu’un modèle réel existe.

Statut et rôle du modèle réel

C'est la toute première chose à supprimer dès qu'un modèle réel (EBM entraîné) existe. `decomposer_en_points` (section 3) doit alors recevoir les vraies contributions log-cotes apprises, à la place de cette liste écrite à la main. Rien d'autre en aval (mapper HTTP, graphique, fiche PDF) n'a besoin de changer : ils affichent déjà correctement n'importe quelle décomposition non vide.

5 Grille de décision (tranche)

Écran(s) concerné(s) : Résultat de scoring (badge de décision, zones colorées de la jauge), écran de paramétrage de la grille.

Implémentation : `backend/solida/domain/rules/grille.py::decider` ; `frontend/lib/scorecard.ts:: trancheDepuisProbabilite` (miroir, même formule)

Variables

Symbole	Signification
p	Probabilité de défaut produite par le modèle

Constantes

Constante	Valeur	Pourquoi cette valeur
$marge$	0,15	Hérite du prototype retiré <code>decision.py</code> – un choix illustratif pour démontrer le mécanisme, sans dérivation actuarielle sur des données réelles.
lgd (perte en cas de défaut)	0,75	Même origine que $marge$.
m_a (multiplicateur accord)	0,6	Même origine.
m_v (multiplicateur vigilance)	1,0	Même origine (le seuil économique lui-même, sans marge de tolérance).
m_e (multiplicateur examen)	1,6	Même origine.

Formule

$$seuil = \frac{marge}{marge + lgd} \quad (6)$$

$$tranche = \begin{cases} \text{ACCORD} & \text{si } p < seuil \times m_a \\ \text{ACCORD_SOUS_CONDITION} & \text{si } p < seuil \times m_v \\ \text{COMITÉ_DE_CRÉDIT} & \text{si } p < seuil \times m_e \\ \text{REFUS} & \text{sinon} \end{cases} \quad (7)$$

Pourquoi cette formule

$seuil$ est le seuil économique classique qui égalise le coût espéré des deux erreurs possibles : refuser un bon dossier (perte de marge) et accorder un mauvais dossier (perte en cas de défaut). Les multiplicateurs créent des zones de tolérance autour de ce seuil plutôt qu'une coupure binaire, pour laisser une marge de jugement humain (accord sous condition, comité de crédit) avant le refus pur. La jauge du résultat de scoring recalcule ces mêmes seuils en score (via la formule de la section 1) pour que les zones colorées affichées correspondent exactement aux frontières de décision – aucune constante supplémentaire n'y est introduite.

Statut et rôle du modèle réel

Le modèle réel améliore uniquement p , l'entrée de **decider**. Il ne calibre jamais *marge*, *lgd* ni les multiplicateurs : ce sont des paramètres de politique de risque de la coopérative, ajustables depuis l'écran de paramétrage de la grille, à rétro-tester sur des cohortes réelles une fois qu'elles existeront.

6 Plafond de crédit progressif

Écran(s) concerné(s) : Résultat de scoring (« Montant recommandé »).

Implémentation : `backend/solida/domain/rules/progressif.py::calculer_plafond, _modulation_risque`

Variables

Symbole	Signification
p	Probabilité de défaut produite par le modèle
M	Montant maximum déjà remboursé par le sociétaire (historique), ou <i>néant</i> pour un primo-emprunteur
D	Montant demandé pour ce nouveau crédit

Constantes

Constante	Valeur	Pourquoi cette valeur
c (coefficient de progression)	1,5	Hérite du prototype retiré <code>decision.py</code> , sans dérivation actuarielle sur des données réelles.
$plancher$	50 000 FCFA	Même origine.
$plafond_produit$	ex. 3 000 000 FCFA	Résolu par produit de crédit (voir section 11), même origine, ajustable par la supervision.
$plafond_primo$	150 000 FCFA	Même origine.
a (base de modulation)	1,3	Même origine.
b (pente de modulation)	2,0	Même origine.
$m_{\min}$	0,4	Même origine.
$m_{\max}$	1,2	Même origine.

Formule

Primo-emprunteur (M absent) :

$$montant_recommandé = \max \left(\min(plafond_primo, D), plancher \right)$$

Sociétaire avec historique :

$$modulation = \text{clip}(a - b \times p, m_{\min}, m_{\max}) \quad (8)$$

$$base = \max(M \times c, plancher) \quad (9)$$

$$montant_recommandé = \max \left(\min(base \times modulation, plafond_produit, D), plancher \right) \quad (10)$$

Pourquoi cette formule

Le crédit progressif (récompenser un historique de remboursement sain par un plafond plus élevé) est un mécanisme d'incitation classique en microfinance, documenté depuis le modèle Grameen (prêt progressif / *step lending*). La modulation par p ajoute une dimension que ce mécanisme historique n'a pas seul : elle évite d'accorder une forte augmentation à un sociétaire

dont le profil de risque s'est dégradé entre deux cycles, même si son historique de remboursement reste bon sur le papier – cohérent avec la pratique de gestion de ligne de crédit bancaire, où le score comportemental borne toujours la décision dans une règle explicite, jamais l'inverse.

Statut et rôle du modèle réel

Le modèle réel améliore uniquement p , l'entrée de *modulation*. Il ne calibre jamais c , *plancher*, *plafond_primo* ni $(a, b, m_{\min}, m_{\max})$: ce sont des paramètres de politique de risque du comité de crédit, à rétro-tester sur des cohortes réelles de remboursement une fois qu'elles existeront en production – un travail distinct de l'entraînement du modèle de score.

7 Palier accessible au prochain cycle

Écran(s) concerné(s) : Résultat de scoring (« Palier suivant accessible »).

Implémentation : `backend/solida/domain/rules/progressif.py::calculer_trajectoire`

Constante

Constante	Valeur	Pourquoi cette valeur
<i>nb_cycles</i>	1	Voir justification détaillée ci-dessous : aucune institution sérieuse ne projette plusieurs cycles à l'avance.

Formule

$$palier = \min (montant_recommandé \times c \times modulation, plafond_produit)$$

où *c* et *modulation* sont identiques à la section 6, *modulation* étant gelée sur la probabilité *p* connue *aujourd'hui*.

Pourquoi une projection sur un seul cycle

Aucune institution de microcrédit sérieuse ne pré-calcule ni ne garantit un plafond à plusieurs cycles d'écart. Les *Guidelines for Establishing and Operating Grameen-Style Microcredit Programs* (Grameen Foundation) l'interdisent même explicitement : chaque proposition de prêt est évaluée et approuvée *au moment du renouvellement*, sur la capacité de remboursement et l'historique du cycle qui vient de s'achever – jamais promise à l'avance – et le code de conduite du gestionnaire de centre y est explicite : « *Does not give any false hope nor lies to members* », « *Strictly keeps any word given to a loanee.* » La gestion de ligne de crédit bancaire suit le même principe (score comportemental réévalué à chaque cycle réel, jamais projeté). Chaque cycle supplémentaire empilerait une hypothèse de bon remboursement sur une hypothèse de risque inchangé, de moins en moins défendable.

Statut et rôle du modèle réel

Identique à la section 6 : seul *p* change avec le modèle réel. La fonction reste paramétrable (*nb_cycles*) si une projection plus longue est un jour explicitement demandée par la coopérative à des fins d'incitation – ce serait alors un choix produit assumé, pas une pratique standard.

8 Conditions de réexamen

Écran(s) concerné(s) : Résultat de scoring (« Conditions de réexamen »), affiché pour toute tranche autre que ACCORD.

Implémentation : `backend/solida/domain/rules/progressif.py::lister_conditions_reexamen`

Constantes – cibles

Constante	Valeur	Pourquoi cette valeur
<i>régularité_cible</i>	75 %	Hérite du prototype retiré <code>decision.py</code> (voir 03-MODELE/09-lecons-prototype-simulateur.md), sans calibration sur des cohortes réelles.
<i>mois_observation</i>	3	Même origine.
<i>ratio_garantie_cible</i>	50 %	Même origine.
<i>endettement_seuil</i>	50 %	Même origine.

Formule (exemple : rattrapage de régularité)

$$mois_à_rattraper = \text{round}((régularité_cible - régularité_actuelle) \times 12)$$

Les autres conditions (garantie, endettement, tendance d'épargne, caution déjà appelée) sont des comparaisons directes au seuil, sans formule dédiée, traduites en phrase actionnable.

Pourquoi ce mécanisme

Un refus ne doit jamais être une porte fermée : chaque condition est un levier concret et vérifiable que le sociétaire peut activer avant le prochain cycle, plutôt qu'un « non » sans recours.

Statut et rôle du modèle réel

Indépendant du modèle de scoring : ces cibles sont des seuils métier, pas des sorties de modèle, et le modèle n'influence pas quelles conditions sont listées – seule la tranche décidée (section 5) détermine si ce bloc s'affiche.

9 Cascade : choix du mode de calcul (socle vs enrichi)

Écran(s) concerné(s) : Résultat de scoring (mention « Score calculé avec l'historique du groupe de caution » ou « profil individuel »).

Implémentation : `backend/solida/domain/rules/cascade.py::determiner_mode`

Constantes

Constante	Valeur	Pourquoi cette valeur
<i>taille_groupe_minimale</i>	3	Aucune source documentée au-delà de l'intuition qu'un groupe de 1 ou 2 membres ne peut pas produire une statistique de groupe fiable.
<i>nb_credits_anterieurs_minimum</i>	3	Arbitrage documenté explicitement dans 03-MODELE/04-cascade-et-cold-start.md : en dessous, le taux de remboursement du groupe repose sur trop peu d'observations et introduirait plus de bruit que de signal.
<i>fraicheur_maximale_jours</i>	7	Aucune source ni calibration documentée pour ce seuil précis – à valider avec la coopérative.

Logique

```
SI les 4 conditions sont reunies
  ALORS mode = enrichi, modele = EBM_enrichi
SINON
  mode = socle_seul, modele = EBM_socle
  motif = raison precise de la bascule
```

Les quatre conditions (appartenance à un groupe, taille, historique de crédits soldés, fraîcheur des features) sont vérifiées dans l'ordre de leur dépendance logique : on ne peut pas juger la taille d'un groupe auquel on n'appartient pas, ni son historique avant d'avoir vérifié sa taille.

Pourquoi ce mécanisme

45 % des dossiers du calibrage retenu sont des primo-emprunteurs : un système qui exigerait un historique de groupe pour scorer serait inutilisable précisément là où le besoin est le plus fort.

Statut et rôle du modèle réel

Cette règle choisit *quel* modèle appeler (socle ou enrichi), elle ne calcule pas elle-même de score. Les trois seuils restent des paramètres de politique ajustables par la coopérative, indépendants de l'implémentation du modèle derrière chacun des deux ports.

10 Tendance de l'épargne

Écran(s) concerné(s) : Dossier sociétaire (badge « en hausse » / « stable » / « en érosion » sur le graphique de trajectoire d'épargne) ; alimente aussi la variable de scoring `tendance_epargne_12m`.

Implémentation : `backend/solida/domain/rules/epargne.py::tendance_depuis_croissance`

Constantes

Constante	Valeur	Pourquoi cette valeur
<code>SEUIL_HAUSSE</code>	10 %	Aucune source. Choisi à la main pour donner un libellé lisible à l'agent au guichet.
<code>SEUIL_EROSION</code>	−5 %	Aucune source. Même choix, non symétrique par intuition (une érosion de l'épargne est jugée significative plus tôt qu'une hausse).

C'est le seul cas de ce document où même l'ordre de grandeur est incertain – contrairement au taux BCEAO de la section 11, il n'existe pas de référence externe pour ce découpage.

Formule

$$tendance = \begin{cases} \text{hausse} & \text{si } croissance_{12m} > SEUIL_HAUSSE \\ \text{érosion} & \text{si } croissance_{12m} < SEUIL_EROSION \\ \text{stable} & \text{sinon} \end{cases}$$

Statut et rôle du modèle réel

Cette fonction alimente à la fois l'affichage et une variable d'entrée du modèle. Un modèle réel entraîné directement sur `croissance_12m` en continu n'a pas besoin de ce découpage en 3 classes, qui perd de l'information par construction. À trancher avec le modèle réel : soit recalibrer ces deux seuils sur des cohortes réelles, soit ne garder ce libellé que pour l'affichage agent et passer la variable continue au modèle.

11 Échéance mensuelle et taux d'endettement

Écran(s) concerné(s) : Formulaire « Nouvelle demande » (« Échéance mensuelle estimée ») ; alimente la variable de scoring `ratio_endettement`.

Implémentation : `backend/solida/domain/rules/echeance.py::calculer_echeance_mensuelle, calculer_taux_endettement`

Variables

Symbole	Signification
D	Montant demandé
n	Durée en mois

Constante

Constante	Valeur	Pourquoi cette valeur
i (taux mensuel)	0,18/12 (18 %/an)	Valeur de démonstration, nettement sous le plafond légal de 24 %/an (TAEG plafond BCEAO pour les SFD, UEMOA, depuis le 1 ^{er} juin 2026), choisie en l'absence d'une table de taux réelle par produit de crédit.

Formule (amortissement à annuité constante)

$$\text{échéance} = D \times \frac{i(1+i)^n}{(1+i)^n - 1}$$

$$\text{taux_endettement} = \frac{\text{charges_mensuelles} + \text{échéance}}{\text{revenu_mensuel}}$$

Pourquoi cette formule

L'amortissement à annuité constante (intérêt dégressif sur le capital restant dû) est la méthode que la réglementation BCEAO impose pour exprimer le TEG/TAEG des crédits dans l'UEMOA, et celle qu'appliquent en pratique les coopératives d'épargne et de crédit togolaises – pas un taux forfaitaire sur le montant initial.

Statut et rôle du modèle réel

Indépendant du modèle de scoring : le taux i dépend d'une vraie table de taux par produit à obtenir de la coopérative, pas d'un entraînement de modèle. Le taux d'endettement calculé ici devient en revanche une variable d'entrée du modèle (section 4), dont le modèle réel apprendra le poids véritable.

12 Politique de mot de passe

Incluse par exhaustivité : ce n'est pas un calcul de crédit, mais c'est une règle à seuils numériques avec sa propre justification, comme tout ce qui précède.

Écran(s) concerné(s) : Connexion, changement de mot de passe.

Implémentation : `backend/solida/domain/rules/mot_de_passe.py::valider_mot_de_passe`

Constantes

Constante	Valeur	Pourquoi cette valeur
LONGUEUR_ MINIMALE	8 caractères	NIST SP 800-63B rev. 4 : 8 caractères est le plancher <i>SHALL</i> (obligatoire), quel que soit le nombre de facteurs d'authentification – 15 n'est qu'une recommandation <i>SHOULD</i> renforcée en l'absence de second facteur. Choix produit : rester praticable pour des agents de coopérative sans authentification multi-facteurs.
LONGUEUR_ MAXIMALE	64 caractères	Même norme : plafond <i>SHALL</i> pour éviter les attaques par déni de service via des mots de passe démesurément longs.

Règle (pas de formule)

Aucune règle de composition (majuscule/chiffre/symbole) : la même norme l'interdit explicitement – ces règles poussent statistiquement vers des mots de passe prévisibles (substitutions convenues, ex. « @ » pour « a »). Rejet uniquement sur la longueur et sur une liste de mots de passe trop courants.

Statut et rôle du modèle réel

Sans rapport avec le modèle de scoring.

Récapitulatif

Calcul	Fichier	Le modèle réel change...	...mais pas
Score (PDO)	<code>scorecard.py</code>	p et les 3 paramètres d'échelle (recalibrage requis)	Les bornes d'affichage 300/850
Invariant de cohérence	<code>scorecard.py</code>	Rien	La tolérance de 1 point
Décomposition en points	<code>scorecard.py</code>	Les contributions f_j (deviennent réelles)	La formule elle-même
Contributions factices	<code>modele_constant.py</code>	Remplacée entièrement	–
Grille (tranche)	<code>grille.py</code>	p	marge, lgd, multiplicateurs
Plafond progressif	<code>progressif.py</code>	p	coefficient, plancher, modulation
Palier au prochain cycle	<code>progressif.py</code>	p	idem + <code>nb_cycles=1</code>
Conditions de réexamen	<code>progressif.py</code>	Rien	Toutes les cibles
Cascade (mode)	<code>cascade.py</code>	Rien (choisit quel modèle appeler)	Tous les seuils
Tendance d'épargne	<code>epargne.py</code>	À trancher (voir section 10)	–
Échéance / endettement	<code>echeance.py</code>	Rien	Taux, indépendant du modèle
Mot de passe	<code>mot_de_passe.py</code>	Rien	Sans rapport avec le modèle